

```
26 // 返回，此时返回目标进程调用该函数前所在地址
27 ret
28 END_FUNC(switch_to_process)
```

图 6.11 展示了 switch_to_process 过程中内核栈的变化过程。假设进程 p0 主动调用 process_waitpid，然后调用 schedule 放弃 CPU 资源，此时调度器选择之前调用 process_sleep 的进程 p1 作为下一个需要执行的进程（即切换的目标进程），那么切换将包含以下步骤。

1. 进程 p0 在 schedule（图中简写为 SC）中使用 bl 指令调用 switch_to_process，此时其返回地址寄存器 X30 会记录 switch_to_process 执行结束后应该返回的地址。

2. switch_to_process 执行与 exception_enter 相似的逻辑，保存处理器上下文。由于该步骤只与内核中的状态相关，因此不需要保存前面提到的特殊寄存器和系统寄存器（SP_EL0、ELR_EL1、SPSR_EL1）。

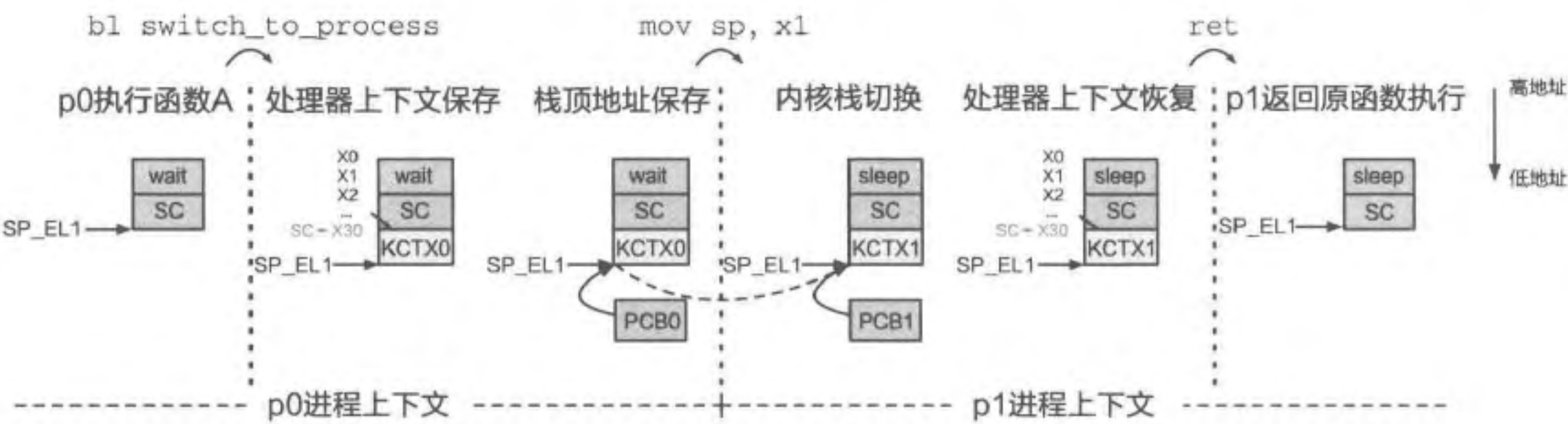


图 6.11 switch_to_process 前后栈结构的变化

3. switch_to_process 将当前的内核栈顶地址保存到原进程 PCB 的 kern_ctx 中，然后将栈寄存器 SP_EL1 切换到目标进程的 kern_ctx，从而完成内核栈切换。

4. 目标进程 p1 执行与 exception_exit 相似的逻辑，恢复通用寄存器，注意此时 p1 之前保存的 X30 也被恢复了。

5. 执行 ret 指令。由于 X30 已被恢复，因此此时会回到目标进程在调用 switch_to_process 之前保存的返回地址继续执行。由于 p1 之前调用了 schedule，那么此时也将回到该函数继续执行，从而完成了切换。

需要留意的是，switch_to_process 只完成了进程内核状态的切换，此时进程在用户态的处理器上下文仍然保存在其内核栈中，仍需要恢复。对于上述例子，当进程 p1 从 process_sleep 中返回时，还需要调用 exception_exit 恢复进程在用户态的处理器上下文，并执行 eret 返回用户态。而对于时钟中断的情况（图 6.8），也需要在调用 schedule 之后再调用 exception_exit 完成处理器上下文恢复，才能正确

地返回用户态执行。

6.5 线程及其实现

本节主要知识点

- 什么是线程？为什么要有线程？
- 线程和进程的区别有哪些？
- 线程是如何实现的？
- 内核态线程和用户态线程都是什么？为什么要分成这两个类别？

6.5.1 为什么需要线程

通过实现进程及其相关接口，操作系统实现了对运行中程序的管理。但是，随着硬件技术的发展，“多核”架构逐渐成为主流，其中的每个 CPU 核心都能独立运行程序，但截至目前介绍的进程只能运行在单个核心之上，因而无法充分利用计算资源。假设小明参加了一场编程比赛，比赛要求编写一个 Web 服务器处理多个客户端发来的请求，在规定时间内处理最多请求的选手取得优胜。为了更快地处理用户请求，小明希望能充分利用自己电脑上的核心进行并行处理，但显然单一进程满足不了这个需求。

那么，小明能否采用创建多个进程的方式来并行处理请求呢？代码片段 6.30 给出了一种在 Linux 操作系统上可能的实现方案。每当获取一个新请求时，程序首先将网络包相关内容放入 packet 结构体中，然后调用 fork 创建新进程并处理网络包。由于新进程的虚拟内存空间中也包含 packet 中的数据，因此可以实现并行处理。

代码片段 6.30 基于 fork 的多进程并行处理程序示例（简化）

```
1 int main(int argc, char *argv)
2 {
3     while (TRUE) {
4         // 配置进程间通信
5         config_ipc(...);
6         // 获取网络包，放入 packet 结构体中
7         struct packet *pkt = recv_packet(...);
8
9         if (fork() == 0) {
10            // 子进程：实际处理网络包
11            struct result* res = handle_packet(pkt);
12            // 告知父进程处理已完成，然后返回
13            notify_complete(res);
14            return 0;
```



```
15     } else {
16         // 父进程：释放 packet 结构体对应的内存
17         free(pkt);
18         struct result *res;
19         if (recv_complete(res)) {
20             // 接收到处理完成的信息，处理结果
21             handle_result(res);
22             // 回收所有退出的子进程
23             while (waitpid(-1, NULL, WNOHANG) > 0);
24         }
25     }
26 }
27 }
```

上述代码虽然能利用多核计算资源，但仍存在以下两方面的问题。

- 进程创建的开销。对于每一个网络包，该程序都需要使用 `fork` 创建一个进程进行处理；而处理完毕后，进程还需要被销毁。由于进程包含的内容较多（PCB、页表、上下文等），创建和销毁都会引入性能开销。如果采用直接从可执行文件创建进程的方式（如 `process_create`），其开销还会进一步包括可执行文件的加载。当网络包处理频率较高时，该开销可能会成为性能瓶颈。
- 如何将处理结果告知父进程？假设子进程在处理完网络包之后，会生成一个处理结果 `res`，然后发给父进程进行收尾工作。但是，由于父进程和子进程处于独立的虚拟地址空间中，因此父进程无法直接访问子进程新创建的 `res`。为了解决这一问题，例子中使用了比较常用的进程间通信机制，通过 `notify_complete` 接口告知父进程自己已经退出，并将 `res` 发给父进程。而父进程则使用 `recv_complete` 确认收到 `res`，并调用 `waitpid` 回收子进程资源（这里的 `notify_complete` 和 `recv_complete` 均为抽象接口，第 8 章将详细介绍进程间通信机制的接口和实现）。不过，进程间通信机制通常都需要内核的协助，因此其开销远高于普通的内存读写操作。

总的来说，由于进程包含的内容较多，且共享数据比较困难，基于多进程的方法会造成性能问题。那么，能否在进程内部引入可并行的多个执行单元，并使其在不同核心上独立执行呢？一方面，由于它们从属于进程，因此在创建和销毁时不涉及进程的所有内容，其开销相对较低；另一方面，由于它们共享同一地址空间，因此可以直接读写彼此的数据，无须使用耗时的进程间通信机制。实际上，目前主流的操作系统已经为这样的执行单元提供了支持，这就是线程（Thread）。

6.5.2 用户视角看线程

小明学习了目前主流的线程库——POSIX 线程库 `pthread` 之后，开始使用线程相关接口编写程序实现并行处理，其简化代码如代码片段 6.31 所示。

代码片段 6.31 利用 POSIX 线程库 pthreads 实现并行数据处理

```
1 // 全局列表，用于存放网络包的处理结果
2 struct list* results;
3
4 void handle_packet(struct packet* pkt)
5 {
6     // 变为分离状态，线程资源自动回收，不需要类似 waitpid 的接口
7     pthread_detach(pthread_self());
8     // 执行处理逻辑，生成结果
9     struct result* res = real_handle(pkt);
10    // 将结果插入全局列表中
11    list_append(results, res);
12    // 释放 packet 结构体对应的内存并退出
13    free(pkt);
14    pthread_exit(NULL);
15 }
16
17 // 由主线程执行 main 函数
18 int main(int argc, char *argv)
19 {
20     while (TRUE) {
21         // 获取网络包，放入 packet 结构体中
22         struct packet *pkt = recv_packet(...);
23         pthread_t new_thread;
24
25         // 创建新线程，并调用 handle_packet
26         pthread_create(&new_thread, NULL,
27                       handle_packet, pkt);
28
29         // 如果全局列表不为空，则进行处理
30         if (results) {
31             handle_results(results);
32             results = NULL;
33         }
34     }
35 }
```

通过使用线程，小明可以解决基于进程的并程序序存在的问题。针对进程创建开销较高的问题，pthread_create 创建的线程依然处于原进程内部，不需要新的页表，因此降低了创建开销。而针对进程间共享数据的问题，由于线程都处于同一地址空间内部，因此可以方便地进行共享数据（如在例子中使用了一个全局列表用于数据共享）。但需要注意的是，由于执行 main 函数的线程（通常称为主线程）会在处理完 results 列表后将其清空，如果此时仍有线程向列表中插入处理结果，可能会因为清空导致遗漏。解决这一问题的方法通常是引入同步原语来同步线程之间的操作，我们将在第 9 章系统介绍相关内容。

从这个例子中，我们还可以发现线程具有与进程相似的管理接口：线程创建和退

出使用的 `pthread_create` 和 `pthread_exit`，与前面介绍的 `process_create` 和 `process_exit` 具有相似的功能。如果要实现线程等待，`threads` 也提供了与 `process_waitpid` 类似的合并 (`join`) 操作，但在该例子中并没有使用，我们将在之后详细介绍。

6.5.3 线程的实现：内核数据结构

由于每个线程都可以独立执行，因此操作系统需要为它们分别维护处理器上下文结构。前面介绍的 PCB 结构只能保存一个处理器上下文，因此很难满足同一进程中多个线程的需要。内核因而为线程设计了专门的数据结构——线程控制块 (Thread Control Block, TCB)。除处理器上下文外，TCB 内还包含以下内容 (如代码片段 6.32 所示)：

- 所属进程。为方便内核管理进程及其包含的线程，TCB 往往包含指向其所属进程 PCB 的指针。
- 内核栈。由于每个线程都是可独立执行的单元，操作系统为它们分别分配了内核栈。
- 线程退出状态。与进程类似，线程在退出时也可以使用整型变量表示其退出状态。
- 线程执行状态。由于线程是调度的基本单元，它也拥有与进程相似的执行状态，调度器通过查看线程的执行状态进行调度。

代码片段 6.32 TCB 结构 (第一版)

```

1 // 一种简单的 TCB 结构实现
2 enum exec_status {NEW, READY, RUNNING, ZOMBIE, TERMINATED};
3
4
5 struct tcb_v1 {
6     // 处理器上下文
7     struct context *ctx;
8     // 所属进程
9     struct process *proc;
10    // 内核栈
11    void *stack;
12    // 退出状态 (用于与 exit 相关的实现)
13    int exit_status;
14    // 执行状态
15    enum exec_status exec_status;
16 };

```

引入线程也使进程的角色发生了变化，因此 PCB 的结构也需要调整。在引入线程之前，进程是操作系统进行资源分配和调度执行的单位；但引入线程之后，线程成为操作系统进行调度执行的单位，而进程主要负责资源管理。因此，与调度执行相关的信

息（处理器上下文和执行状态）都从 PCB 中移入 TCB，而与资源管理相关的信息（如虚拟地址空间）依然保存在 PCB 中。另外，内核栈和退出状态也与执行相关，因此进程不再维护，改为由线程（TCB）维护。如代码片段 6.33 所示，支持线程的 PCB 结构与之前的结构相比，其不再包含处理器上下文 ctx、内核栈 stack、退出状态 exit_status 和执行状态 exec_status，但保留了虚拟地址空间 vmSPACE，且加入了线程列表 threads 和线程总数 thread_cnt 用于维护线程相关信息。

代码片段 6.33 PCB 结构（第六版）：加入线程支持

```
1 // 支持线程之后的 PCB 结构实现
2 struct process_v6 {
3     // 虚拟地址空间
4     struct vmSPACE *vmSPACE;
5     // 进程标识符
6     int pid;
7     // 子进程列表
8     pcb_list *children;
9     // 包含的线程列表
10    tcb_list *threads;
11    // 包含的线程总数
12    int thread_cnt;
13 };
```

练习

6.8 引入线程对微内核的 PCB 有什么影响？

6.5.4 线程的实现：管理接口

与进程的管理接口相似，线程的管理接口也提供了类似的功能：创建、退出、等待等。但由于线程是更加轻量级的单元，因此其接口与进程有所差别，本节将分别进行介绍，并结合 pthreads 进行说明（本节介绍的接口定义如代码片段 6.34 所示）。

代码片段 6.34 pthreads 部分重要的线程管理接口：创建、退出、等待（合并）和分离

```
1 // 线程创建
2 int pthread_create(pthread_t *restrict thread,
3                    pthread_attr_t *restrict attr,
4                    void *(*start_routine)(void *),
5                    void *restrict arg);
6 // 线程退出
7 void pthread_exit(void *retval);
8 // 线程等待 / 合并
```

```

9 int pthread_join(pthread_t thread, void **retval);
10 // 线程分离
11 int pthread_detach(pthread_t thread);

```

线程创建

Linux 中的经典进程创建包含两个接口：fork 创建一份原进程的拷贝，而 exec 载入新的可执行文件并执行。由于进程包含的内容较多（可执行文件、堆栈结构、处理器上下文等），基于拷贝的 fork 可以使进程的创建过程得到简化。而线程包含的内容本来就比较少，也不需要载入新的可执行文件，因此可直接提供 thread_create 接口，同时完成创建和执行功能。

从代码片段 6.35 中的伪代码可以看出，线程创建的过程与前面介绍的 process_create 比较相似，但进行了简化。首先，线程创建需要分配管理线程的数据结构——TCB，类似 process_create 中的 PCB 创建。之后，线程需要初始化内核栈和处理器上下文（主要是 PC 和 SP）并设置参数，这部分在 process_create 中也有体现。除此之外，线程还需要维护与所属进程之间的关系，便于操作系统的后续管理。不过，thread_create 不包括虚拟地址空间初始化、可执行文件载入、用户栈分配等过程，这也使得线程的创建要比进程简单得多。

代码片段 6.35 线程创建的伪代码实现

```

1 int thread_create(u64 stack, u64 pc, void *arg)
2 {
3     // 创建一个新的 TCB，用于管理新线程
4     struct tcb *new_thread = alloc_thread();
5     // 内核栈初始化
6     init_kern_stack(new_thread->stack);
7
8     // 创建线程的处理器上下文
9     new_thread->ctx = create_thread_ctx();
10    // 初始化线程的处理器上下文（主要包括用户栈和 PC）
11    init_thread_ctx(new_thread, stack, pc);
12
13    // 维护进程与线程之间的关系
14    new_thread->proc = curr_proc;
15    add_thread(curr_proc->threads, new_thread);
16    // 设置参数
17    set_arg(new_thread, arg)
18    // 返回
19 }

```

对于线程创建，POSIX 提供的接口为 pthread_create，接收四个参数。虽然其接口与 thread_create 有一定差别，但其核心功能基本相同。start_routine 是线程创建后执行的起点，因此对应了 PC 寄存器的初始值，而线程的用户栈起始地址等信

息则保存在 `attr` 中。

练习

6.9 基于拷贝的 `fork` 长久以来一直是创建进程的主要方法，那为什么不采用类似的方法创建线程呢？

线程退出

进程退出使用的接口 `process_exit` 包含的逻辑比较复杂，因为与进程相关的内容都需要销毁。与之相比，线程退出只需销毁与之相关的内容。从代码片段 6.36 可以看出，线程实际上只需要销毁处理器上下文即可。注意，为了方便内核管理，我们引入 `curr_thread` 指向当前正在运行的线程的 TCB。与 `process_exit` 类似，为了支持监控功能，内核不应该销毁 TCB，而是需要在 TCB 中存储线程退出时的状态。此外，进程还需要移除当前终止的线程。如果当前线程是进程拥有的最后一个线程，那么线程退出就意味着进程也不再执行了。因此，内核会销毁当前 TCB（因为该线程不会再被监控），还会调用 `process_exit` 销毁当前进程。最后，`thread_exit` 也需要调用与 `process_exit` 类似的 `schedule` 函数，选择下一个需要执行的线程。

代码片段 6.36 线程退出的伪代码实现（第一版）

```
1 void thread_exit_v1(int status)
2 {
3     // 获取当前线程的所属进程
4     struct process *curr_proc = curr_thread->proc;
5     // 存储返回值
6     curr_thread->exit_status = status;
7     // 销毁处理器上下文
8     destroy_thread_context(curr_thread->ctx);
9
10    // 从进程的列表中移除当前线程
11    remove_thread(curr_proc->threads, curr_thread);
12    curr_proc->thread_cnt--;
13    // 如果进程中不再包含任何线程，销毁 TCB 和进程
14    if (curr_proc->thread_cnt == 0) {
15        destroy_thread(curr_thread);
16        process_exit(curr_proc);
17    }
18    // 告知内核选择下个需要执行的线程
19    schedule();
20 }
```

`pthread`s 中的线程退出接口 `pthread_exit` 与代码片段 6.35 类似，只是放宽了返回值的类型。

线程等待与分离

与进程等待功能相似的线程管理接口是合并操作 join。线程可以调用 `thread_join` 并指定需要监控的线程，当 `thread_join` 被调用后，线程会等待其指定的线程退出。当 `thread_join` 返回时，指定线程的返回值也会传给调用 `thread_join` 的线程。

如代码片段 6.37 所示，`thread_join` 与 `process_waitpid` 以及 POSIX 中的 `waitpid` 相比有一定的相似之处：`thread_join` 也提供等待机制，并且也负责一部分资源回收工作。如果线程退出且没有其他线程调用 `thread_join`，那么它将转变为僵尸状态。与 `wait` 相比，`thread_join` 的功能较为单一，它只监控指定线程的退出事件，而 `wait` 及 `waitpid` 可以监控任意子进程的多种事件。这也是线程监控的接口取名为 `join` 的原因：当 `join` 调用返回之后，两个线程将“合并”为一个。与用于线程退出的接口类似，`pthread` 为线程监控提供的接口 `pthread_join` 只是扩展了返回值的类型，其他内容与代码片段 6.37 差别不大。

代码片段 6.37 线程监控的伪代码实现

```

1 void thread_join(struct tcb *thread, int *status)
2 {
3     // 等待，直到线程退出
4     while (thread->exec_status != ZOMBIE)
5         schedule();
6     // 获取指定线程的返回值，存储到参数中
7     *status = thread->status;
8     // 销毁指定线程的 TCB
9     destroy_thread(thread);
10 }
```

`thread_join` 要求线程等待其他线程退出并回收其资源。但在前面小明的例子中，由于 Web 服务器要不停接收请求和处理结果，可能抽不出时间来调用 `thread_join`。针对这种场景，线程提供了分离接口，允许分离线程自行回收资源。代码片段 6.38 展示了分离接口 `thread_detach` 的实现方式。为了支持线程分离，TCB 内部需要维护与分离相关的状态 `is_detached`。而当线程退出时，如果发现线程处于分离状态，则可以直接回收其 TCB。如果一个线程处于分离状态，则 `join` 操作对其无效。

代码片段 6.38 线程分离的伪代码实现及其对 TCB 结构 / 线程退出实现的影响

```

1 void thread_detach(struct tcb *thread)
2 {
3     // 将 thread 的状态改为分离
4     thread->is_detached = TRUE;
5 }
6
7 struct tcb_v2 {
8     // 处理器上下文
9     struct context *ctx;
```

```

10 // 所属进程
11 struct process *proc;
12 // 内核栈
13 void *stack;
14 // 退出状态 (用于与 exit 相关的实现)
15 int exit_status;
16 // 执行状态 (省略了枚举类型的定义)
17 enum exec_status exec_status;
18 // 分离相关
19 bool is_detached;
20 };
21
22 void thread_exit_v2(int status)
23 {
24     // 获取线程的所属进程
25     struct process *curr_proc = curr_thread->proc;
26     // 存储返回值
27     curr_thread->status = status;
28     // 销毁上下文
29     destroy_thread_context(curr_thread->ctx);
30     // 从进程的列表中移除当前线程
31     remove_thread(curr_proc->threads, curr_thread);
32     curr_proc->thread_cnt--;
33
34     // 如果进程中不再包含任何线程, 则销毁 TCB 和进程
35     if (curr_proc->thread_cnt == 0) {
36         destroy_thread(curr_thread);
37         process_exit(curr_proc);
38     }
39     // 如果是分离线程, 则直接销毁其 TCB
40     if (thread->is_detached)
41         destroy_thread(curr_thread);
42     // 告知内核选择下个需要执行的线程
43     schedule();
44 }

```

读者可能会有疑问, 为什么线程可以自行回收资源, 但进程需要由父进程帮助回收呢? 这是因为进程与线程之间的关系存在差别。进程间的关系是分层的, 依照创建与被创建关系构成树状结构, 根节点 (父进程) 有管理叶节点 (子进程) 的职责。而线程间的关系是扁平的, 线程只存在创建先后的关系, 并不存在“父子线程”的关系, 因此每个线程都可以调用 `detach` 自行回收资源, 也可以调用 `join` 回收同一进程中任意线程的资源 (当然, 该线程不能是分离线程)。

小知识: fork 与多线程

如果一个多线程的程序调用 `fork`, 会发生什么呢?

考虑到 `fork` 的语义是创建与父进程一模一样的进程拷贝, 一种直观的实现方式

是拷贝父进程中所有的线程。但是，由于每个线程都是相对独立的，在 `fork` 之后还会继续执行，这种方法可能会导致程序表现出意料之外的行为。比如在线程 1 调用 `fork` 时，线程 2 正要调用 `write` 操作修改一个文件。由于线程 2 在 `fork` 时被拷贝到子进程中，因此这个写操作将会被执行两次（分别来自父进程和子进程的线程 2），这可能并不是应用期望的结果。为了避免这种情况，应用需要在调用 `fork` 前限制其他线程的执行（如不允许修改文件），从而使编码变得复杂。因此，最终 POSIX 标准并没有支持这种实现，而是选取了一种更加简单的实现。

POSIX 规定，`fork` 生成的子进程中只保留一个线程，它是父进程中调用 `fork` 的线程的拷贝。对于父进程中的其他线程，它们在内存中的所有状态仍然会被拷贝到子进程中，但线程本身并不会被拷贝，这样就避免了上述重复操作的问题。但这种实现也有问题：由于线程通常不会访问其他线程私有的内存区域（如线程栈），这些内存存在拷贝到子进程后实际上并不会被使用，造成了内存浪费，且只有在进程退出时才能回收。由于两种 `fork` 的实现都存在问题，POSIX 建议程序员在调用 `fork` 拷贝包含多个线程的进程之后，尽快调用 `exec` 以消除因共享状态带来的影响。

6.5.5 线程切换

由于线程取代进程成为内核调度的基本单位，因此分时执行中切换的单位也变为了线程。6.4 节已经对进程切换做了比较详细的介绍，而线程切换的过程也与进程切换大致相同，其工作流包括：原线程进入内核态→保存原线程的处理器上下文→切换线程上下文→恢复目标线程的处理器上下文→目标线程返回用户态。这里同样出现了两个上下文的概念。与进程上下文类似，线程上下文也是操作系统提供的软件概念，内核也会提供一个指向当前运行线程 TCB 的指针（例如之前引入的 `curr_thread`）。

线程切换与进程切换最大的差别出现在上下文切换部分：进程的上下文切换包含虚拟地址空间和内核栈的切换；而在线程的上下文切换中，如果两个线程从属于同一个进程，那么就不会进行虚拟地址空间的切换，只进行内核栈切换。因此，线程切换的步骤相比进程有时会略少，这也是线程轻量化特性的一种体现。

6.5.6 内核态线程与用户态线程

6.5.4 节介绍了 `pthread_exit` 的接口，它与进程退出接口 `exit` 的最大差别是参数类型：`pthread_exit` 可以接收任意类型的参数作为线程的返回值。这就带来了一个严重的问题：在 `thread_exit` 的伪代码实现中，该返回值是存储在内核的 TCB 中的，如果返回值可以是任意大小的数据类型，那么 TCB 的内存开销也可能会大幅上升，增大内核的内存压力。如果内核占用的内存资源过多，那么应用执行也会受到影响，因此由内核管理该返回值是不合适的。

那么，`pthread`s 该如何支持任意类型的返回值呢？注意到 `pthread`s 提供的 POSIX 接口并不是系统调用，而是在用户态运行的线程库函数。因此，`pthread`s 可以在用户态设计相应的数据结构，用于保存线程的返回值。

如图 6.12 所示，`pthread`s 与线程相关的数据结构实际上被拆分成两部分：内核结构（`tcb`）依然保存与线程相关的重要信息（如所属进程和处理器上下文），这些信息不可被用户直接访问；其他信息则保存在用户态的数据结构（`pthread`）中，内核并不知晓这些数据与该线程的关系。

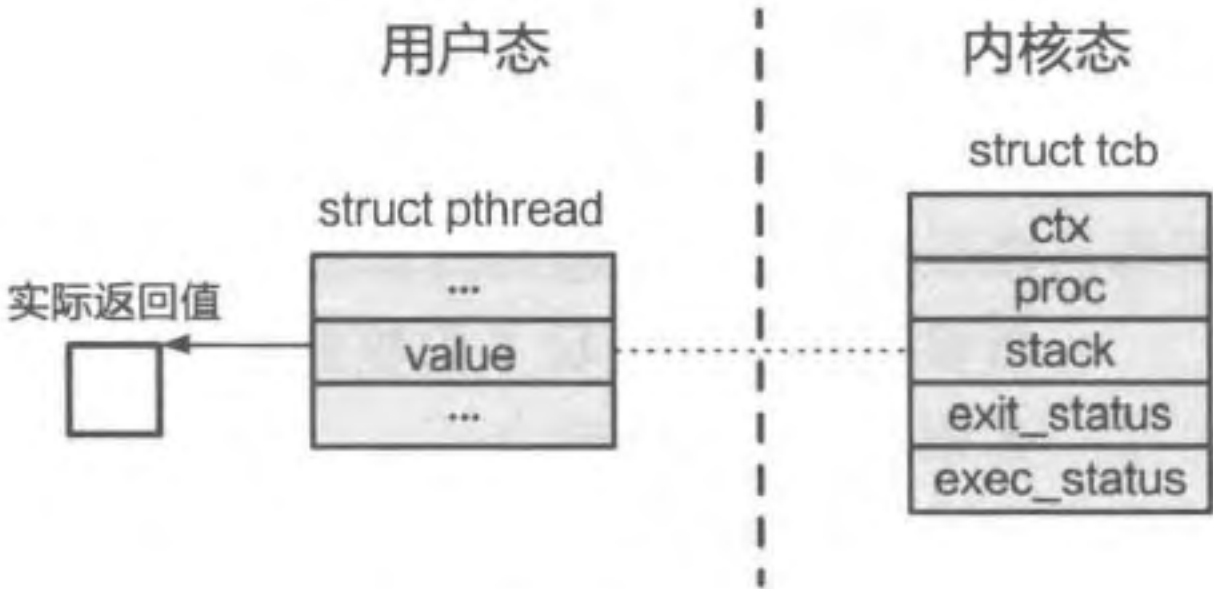


图 6.12 pthreads 与线程相关的数据结构分为两部分

由于线程在内核态和用户态的执行实际上拥有较强的独立性，我们可以将其视为两类线程。内核态线程（Kernel Level Thread）是由内核创建并直接管理的线程，内核维护了与之对应的数据结构——TCB。而用户态线程（User Level Thread）是由用户态的线程库创建并管理的线程，其对应的数据结构保存在用户态，内核并不知晓该线程的存在，也不对其进行管理。由于内核只能管理内核态线程，其调度器（第 7 章会详细介绍）只能对内核态线程进行调度，因此用户态线程如果想要执行，就需要“绑定”到相应的内核态线程才能执行。

读者可能会对这“两类线程”感到疑惑：根据第 3 章的介绍，`pthread`s 作为线程库只是提供了更加方便的抽象，其背后还是对应着由内核管理的线程，为什么要将其拆分为内核态线程和用户态线程呢？这个问题在 `pthread`s 场景下确实不好回答，因为它的用户态线程与内核态线程始终是一一对应的。但在本节中读者将发现，用户态线程与内核态线程的关系还可以更加复杂，而刻画用户态线程与内核态线程关系的方法就称为多线程模型（Multithreading Model）。一般来说，多线程模型分为以下三类（如图 6.13 所示）。

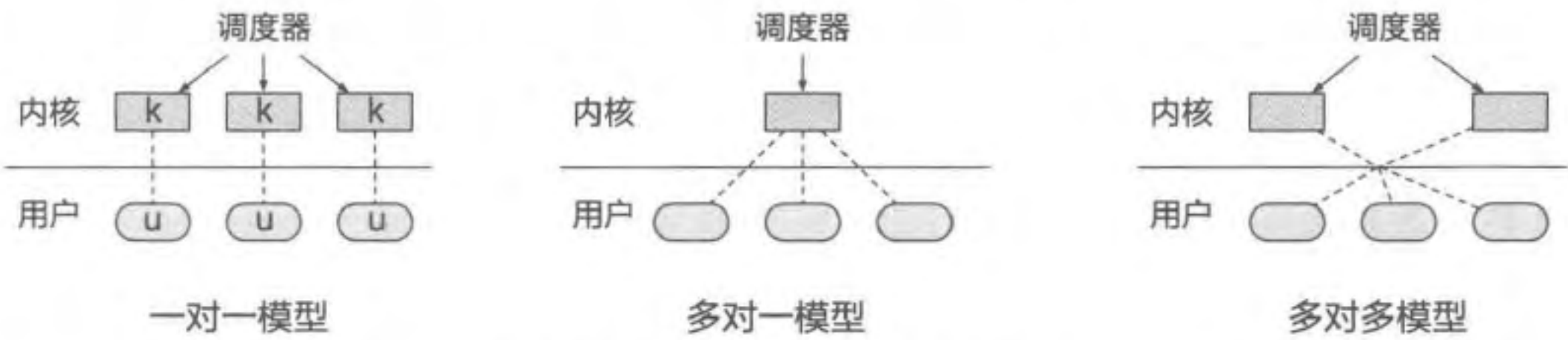


图 6.13 三种不同的多线程模型

- 一对一模型。一对一模型是现在最为常见的多线程模型，在这种模型中，每个用户态线程都对应一个单独的内核态线程。因此，当内核态线程被内核调度执行时，其对应的用户态线程就会被执行，两者关系非常紧密，看起来就像是同一个线程

的两个部分。pthreads 采取的就是这种一对一模型。

- 多对一模型。多对一模型允许将多个用户态线程映射到单一的内核态线程。如图 6.13 所示，在这种模型下，进程实际上只分配一个内核态线程，其多线程环境完全由用户态的线程库实现。比较有名的使用多对一模型的线程库是 Green Thread，它最初是由 Sun 公司为其 Java 应用开发的。由于当时的计算机资源（如内存）有限，创建内核态线程会给系统带来较大的压力。与内核态线程相比，Green Thread 创建的线程状态及其执行情况完全由用户态线程库管理，不需要内核参与，从而减轻了内核的压力。Green Thread 一词后来也成为用户态线程的代名词，可见其影响力。
- 多对多模型（ $M:N$ ）。多对一模型有一个显著的问题：由于进程中只包含一个内核态线程，因此同时只能被调度到一个 CPU 上执行，这导致应用的可扩展性受限。针对这一问题，Sun 公司又提出了多对多模型，它允许应用自定义进程中包含的内核态线程数量（例如可以和 CPU 数量保持一致），之后由线程库将用户态线程映射到不同的内核态线程执行。这种模型既突破了多对一模型的可扩展性限制，又减少了一对一模型存在的资源开销。

总的来说，多对多模型和多对一模型最初都是针对资源受限场景提出的，可以在占用较少内核资源的情况下创建大量线程。除此之外，用户态线程的创建、切换、销毁等功能都不需要进入内核，因此时延较低。但是，用户态线程的大量引入也会让线程管理变得更加复杂。随着计算机资源的丰富及 pthreads 的普及，多对多和多对一模型逐渐被更加简洁的一对一模型取代，就连 Sun 公司的 Solaris 操作系统也在第 9 版之后改为了一对一模型。不过，多对多 / 多对一模型并没有完全消亡，近年来随着纤程的发展，它们又迸发了新的活力。

6.6 纤程

本节主要知识点

- 为什么对纤程的需求在近年来越发强烈？
- 什么是纤程？如何使用纤程？
- 纤程如何进行切换？
- 纤程与进程、线程的区别有哪些？

以 pthreads 为代表的一对一线程模型在用户态线程和内核态线程之间建立了紧密的联系，用户态线程可以利用内核支持完成多种功能（如第 9 章将介绍的线程同步），因而得到了比较广泛的使用。但是，随着计算机的发展，应用类型和需求渐趋多样化，一对